

Report

Part 1

We were asked to develop refactor our existing java classes and add a new one (`StochasticGradientDescent`). Refactoring consisted in using abstraction to improve code readability and reuse. We had to add new classes such as `RawDataset` and `GradientDescent`. Also, for this Lab we had to add method overloading for method `transform()`, which, even though, is implemented in the abstract `Dataset` class, it is really used in `StandardizedDataset`. Our main purpose of this Lab was to implement abstract programming to remove redundant methods while inheriting (for example computing mean and standard deviation in `StandardizedDataset` class is redundant, but due to inheritance from `Dataset` class it was possible). Apart from the implementation of abstraction, we added a new algorithm with class `StochasticGradientDescent`. The main differences from the `GradientDescent` class are the way of stopping teaching the model (which is done using iterations and not `stopping_criterion`) and training the model on a reduced number of records from the dataset, which is implemented using the `batch_size` and `random` attributes. Otherwise, it is a complete copy of the `GradientDescent` class.

Part 2

In this Lab, the only alternative solution that was discussed consisted in how to transform the `Vector v` in the argument of the `predict` method, if the dataset was an instance of `StandardizedDataset`: whether to create a “dummy” `Record` and use the existing `transform` method, or to create another `transform` method whose argument and output are vectors, instead of records. In the end, we decided to go with the option that, for us, is “cleaner” (since no additional instances have to be created) and more object-oriented (since we take advantage of overloading): creating another `transform` method, to only work with the existing `Vector` and avoid creating a new `Record`.

When it comes to related theoretical concepts, we mainly worked with abstract classes (`Dataset` and `Algorithm`), which implies the use of abstract methods and the overriding of these methods. In addition, as it was mentioned before, we applied overloading in the `transform` method.

Part 3

When it came to testing the correct implementation of the abstraction and `StochasticGradientDescent`, most of the methods could be tested by checking that the prediction of instance `SupervisedLearner` trained on specific dataset is close enough to the expected value. We had to train 3 models: 2 on `RawDataset` and 1 on `StandardizedDataset` to ensure that the methods `transform` and `output` are correctly implemented. The first two models used were distinguished by the algorithm they used

(either `GradientDescent` or `StochasticGradientDescent`). The last one was trained on `StandardizedDataset` using `GradientDescent` algorithm. All three test cases gave positive results which were very close to the expected value.

We didn't find any difficulties in the implementation of this Lab.